

Manual Operativo de Datos

Esquema de Airtable y estructuras JSON de integración — Ecosistema de Atención al Cliente con Memoria (Estudio Contable)

Este documento describe la “memoria” del sistema: cómo están organizadas las tablas de Airtable, cómo se vinculan entre sí, y qué estructura JSON viaja entre cada integración del flujo de n8n (Gmail → Airtable → Gemini API → Slack → Gmail). Todos los bloques JSON de este documento son JSON válido, verificado programáticamente antes de incluirse aquí.

1. Base de Airtable: estructura de tablas vinculadas

La base “Estudio Contable — Atención al Cliente” tiene tres tablas relacionadas. Esto evita datos aislados: cada consulta queda vinculada a un cliente, y cada error queda vinculado a la consulta que lo originó.

Tabla: Clientes

Campo	Tipo	Notas
Nombre_Razon_Social	Texto corto	—
Email	Texto corto	Clave de búsqueda desde el trigger de Gmail
Telefono	Texto corto	—
Consultas	Link a tabla Consultas	Relación 1→N, poblada automáticamente

Tabla: Consultas

Campo	Tipo	Notas
Cliente	Link a tabla Clientes	1 consulta → 1 cliente
Asunto	Texto largo	—
Cuerpo_Original	Texto largo	Cuerpo del email tal como llega
Estado	Single select	Pendiente / No identificado / Procesado por IA / Respondido / Rechazado
Categoria_IA	Single select	Clasificación que asigna Gemini: Vencimiento / Comprobante / Consulta general / Otro
Borrador_Respuesta_IA	Texto largo	Salida del nodo de IA, antes de HITL
Respuesta_Final_Enviada	Texto largo	Lo enviado realmente (puede diferir del borrador si el humano editó)
Fecha_Aprobacion	Fecha/hora	—
Errores	Link a tabla Errores	Vacío si no hubo error

Tabla: Errores

Campo	Tipo	Notas
Consulta_Relacionada	Link a tabla Consultas	—
Tipo_Error	Single select	API_IA / Dato_Faltante / Timeout / Otro
Detalle_Error	Texto largo	Mensaje/código devuelto por el nodo que falló
Estado_Resolucion	Single select	Pendiente / Resuelto

Las tablas Clientes → Consultas → Errores forman una cadena de trazabilidad: desde un registro de error siempre se puede subir hasta el cliente afectado. El Dashboard de Control (documento 5) se construye con vistas y campos de resumen (rollup/count) sobre estas mismas tablas, no como una cuarta tabla separada.

2. Esquemas JSON de las integraciones

Cada flecha del diagrama de arquitectura corresponde a un payload JSON real dentro de n8n. Se documentan los siete puntos de integración, en el orden en que ocurren dentro del flujo. Los ejemplos usan valores ilustrativos.

2.1 — Trigger: Gmail (nodo Gmail Trigger)

Salida simplificada que recibe el siguiente nodo:

```
{
  "from": "cliente@dominio.com",
  "subject": "Consulta sobre vencimiento de IVA",
  "textPlain": "Hola, queria saber cuando vence mi proxima presentacion.",
  "messageId": "18f2a1b@mail.gmail.com",
  "receivedDate": "2026-09-10T13:42:00-03:00"
}
```

2.2 — Airtable: búsqueda de cliente

Request (nodo Airtable → Search, filtro por email):

```
{
  "table": "Clientes",
  "filterByFormula": "{Email} = 'cliente@dominio.com'",
  "maxRecords": 1
}
```

Response:

```
{
  "records": [
    {
      "id": "recA1B2C3",
      "fields": {
        "Nombre_Razon_Social": "Panaderia del Centro SRL",
        "Email": "cliente@dominio.com",
        "Telefono": "3534123456"
      }
    }
  ]
}
```

2.3 — Airtable: historial de consultas (memoria)

```
{
  "records": [
    {
      "fields": {
        "Categoria_IA": "Vencimiento",
        "Respuesta_Final_Enviada": "Le confirmamos que el vencimiento es el..."
      }
    }
  ]
}
```

Este historial (últimas 3–5 consultas del cliente) se inserta como contexto variable en el prompt de Gemini: es la “memoria” que da continuidad a la atención.

2.4 — Gemini API: request (nodo HTTP Request en n8n)

Se mantiene “lo fijo al principio, lo variable al final” por prolijidad y legibilidad del prompt, aunque en Gemini esto no activa caching automático a este tamaño: el caching implícito de Gemini 3.6 Flash recién arranca a partir de 4.096 tokens de prefijo (ver Matriz de Costos, documento 3) y el bloque fijo de este sistema ronda los 600. La estructura del body es la de la Gemini API:

system_instruction + contents[], no messages[] como en otros proveedores.

```
{
  "system_instruction": {
    "parts": [
      {
        "text": "Asistente de atencion al cliente del Estudio Contable X.
        Clasifica la consulta (Vencimiento, Comprobante, Consulta_general,
        Otro) y redacta una respuesta breve en espanol rioplatense. No
        inventes fechas ni montos. Devuelve solo JSON con las claves
        categoria y respuesta_borrador."
      }
    ]
  },
  "contents": [
    {
      "role": "user",
      "parts": [
        {
          "text": "CLIENTE: Panaderia del Centro SRL. HISTORIAL: ver 2.3.
          CONSULTA: 'Hola, queria saber cuando vence...'"
        }
      ]
    }
  ],
  "generationConfig": {
    "maxOutputTokens": 600
  }
}
```

2.5 — Gemini API: response

```
{
  "candidates": [
    {
      "content": {
        "role": "model",
        "parts": [
          {
            "text": "{\"categoria\": \"Vencimiento\",
            \"respuesta_borrador\": \"Hola! Le confirmamos que...\"}"
          }
        ]
      }
    ],
    "finishReason": "STOP"
  ],
  "usageMetadata": {
    "promptTokenCount": 612,
    "candidatesTokenCount": 118,
    "totalTokenCount": 730,
    "cachedContentTokenCount": 0
  }
}
```

cachedContentTokenCount da 0 en este caso puntual: confirma que, con un bloque fijo de ~600 tokens, la llamada queda por debajo del mínimo cacheable de Gemini y se factura completa (ver Matriz de Costos, documento 3).

2.6 — Slack: aprobación HITL (nodo nativo "Send and Wait")

A diferencia de las otras integraciones, este paso no arma el JSON de Slack a mano: usa la operación "Send and Wait" del nodo de Slack en n8n. n8n arma el Block Kit internamente (mensaje + botones) a partir de estos parámetros, y pausa la ejecución en el mismo lugar hasta que alguien responde — no hace falta un endpoint de Slack aparte ni reconstruir el contexto en una segunda ejecución.

```
{
```

```

"operation": "sendAndWait",
"channel": "aprobaciones-ia",
"message": "Nueva consulta para aprobar\nCliente: Panaderia del Centro
  SRL\nCategoria: Vencimiento\n\nConsulta original:\n[texto de la
  consulta]\n\nBorrador de respuesta:\n[texto del borrador]",
"approvalOptions": {
  "approvalType": "double",
  "approveLabel": "Aprobar y enviar",
  "disapproveLabel": "Rechazar"
}
}

```

Al continuar la ejecución, el nodo entrega el resultado en `{"data": {"approved": true|false}}` — el router siguiente (¿Aprobado por humano?) evalúa directamente ese campo booleano.

2.7 — Airtable: actualización final (nodo Airtable → Update)

```

{
  "table": "Consultas",
  "id": "recCons123",
  "fields": {
    "Estado": "Respondido",
    "Respuesta_Final_Enviada": "Hola! Le confirmamos que...",
    "Fecha_Aprobacion": "2026-09-10T13:47:22-03:00"
  }
}

```

El archivo técnico completo (Estudio_Contable_Atencion_Cliente_FINAL.json) incluido en el repositorio contiene la definición real de los nodos y sus conexiones, validada con ejecuciones reales.